Contents

1 Perform threat hunting (20–25%) 4
1.1 Detect threats by using Microsoft Defender XDR 4
1.1.1 Identify the appropriate table to use in a KQL query . . . 4
1.1.1.1 Alertas 4
1.1.1.2 Defender for Endpoint 4
1.1.1.3 Defender for Office 365 5
1.1.1.4 Identidades 5
1.1.1.5 Defender for Cloud Apps 5
1.1.1.6 Microsoft Graph 6
1.1.1.7 Defender for Cloud 6
1.1.1.8 Cómo elegir la tabla en el examen 6
1.1.1.9 Schema reference 7
1.1.2 Identify threats by using Kusto Query Language (KQL) . 7
1.1.2.1 where 8
1.1.2.2 Tiempo 8
1.1.2.3 Operadores de strings 9
1.1.2.4 Case sensitivity 9
1.1.2.5 project 9
1.1.2.6 extend 10
1.1.2.7 sort/order y top 10
1.1.2.8 limit / take 10
1.1.2.9 summarize 11
1.1.2.10 bin 11
1.1.2.11 join 12
1.1.2.12 let 12
1.1.2.13 Datos dynamic 13
1.1.2.14 Patrón de hunting recomendado 13
1.1.2.15 Ejemplo completo de hunting 14
1.1.3 Create Advanced Hunting queries 14
1.1.3.1 Crear una query 14
1.1.3.2 Schema y ayuda 15
1.1.3.3 Go hunt 15
1.1.3.4 Guided mode 16
1.1.3.5 Guardar consultas 16
1.1.3.6 Múltiples consultas 16
1.1.3.7 Validar resultados 17
1.1.3.8 Flujo recomendado para SC-200 17
1.1.3.9 Escenarios típicos de examen 17
1.1.4 Interpret threat analytics in Microsoft Defender XDR . . 18
1.1.4.1 Threat analytics dashboard 18
1.1.4.2 Threat analytics report 18
1.1.4.3 Overview 19

1.1.4.4 Related incidents . . . 19
1.1.4.5 Impacted assets . . . 19
1.1.4.6 Endpoints exposure . . . 19
1.1.4.7 Recommended actions . . . 20
1.1.4.8 Analyst report . . . 20
1.1.4.9 Indicators . . . 20
1.1.4.10 Escenarios típicos de examen . . . 21
1.1.5 Create hunting graphs, including blast radius . . . 21
1.1.5.1 Crear un hunting graph . . . 22
1.1.5.2 Explorar el grafo . . . 23
1.1.5.3 Blast radius . . . 23
1.1.5.4 Interpretar Blast radius . . . 23
1.1.5.5 Uso operativo . . . 24
1.1.5.6 Escenarios típicos de examen . . . 24
1.1.6 Analyze relationships between entities by using Sentinel Graph . . . 24
1.1.6.1 Graph Query Language . . . 25
1.1.6.2 Identity attack graph . . . 25
1.1.6.3 Explorar relaciones . . . 26
1.1.6.4 Custom graphs . . . 26
1.1.6.5 Escenarios típicos de examen . . . 27
1.2 Detect threats by using the Microsoft Sentinel platform . . . 27
1.2.1 Create and monitor hunting queries . . . 27
1.2.1.1 Tipos de hunting queries . . . 28
1.2.1.2 Crear una hunting query . . . 28
1.2.1.3 Ejecutar y monitorizar queries . . . 29
1.2.1.4 View query results . . . 29
1.2.1.5 Bookmarks . . . 30
1.2.1.6 Hunts . . . 30
1.2.1.7 Convertir hallazgo en detección . . . 31
1.2.1.8 Escenarios típicos de examen . . . 31
1.2.2 Create and manage KQL jobs in Data lake . . . 32
1.2.2.1 Crear un KQL job . . . 32
1.2.2.2 Destination table . . . 33
1.2.2.3 Promote to Analytics tier . . . 33
1.2.2.4 One-time vs. Scheduled . . . 33
1.2.2.5 Schema . . . 34
1.2.2.6 Casos típicos de examen . . . 34
1.2.3 Create and manage Summary rule tables for querying . . . 34
1.2.3.1 Casos de uso . . . 35
1.2.3.2 Crear una Summary Rule . . . 35
1.2.3.3 Ejemplo . . . 36
1.2.3.4 Summary Rule vs. KQL Job . . . 36
1.2.3.5 Escenarios típicos de examen . . . 36
1.2.4 Hunt for threats by using Notebooks, including connection to the Sentinel MCP Server . . . 37

1.2.4.1 Notebooks 37
1.2.4.2 MSTICPy 38
1.2.4.3 Sentinel Data Lake notebooks 38
1.2.4.4 Microsoft Sentinel MCP Server 39
1.2.4.5 Data Exploration collection 39
1.2.4.6 Conexión al MCP Server 40
1.2.4.7 Notebooks + MCP 40
1.2.4.8 Escenarios típicos de examen 41

1 Perform threat hunting (20–25%)

1.1 Detect threats by using Microsoft Defender XDR

1.1.1 Identify the appropriate table to use in a KQL query

Advanced Hunting utiliza un conjunto de tablas especializadas según el tipo de telemetría que se quiere investigar.

La regla práctica es:

Primero identificar qué actividad buscas; después elegir la tabla

1.1.1.1 Alertas

Tabla	Uso
`AlertInfo`	Información de la alerta: título, severidad, categoría, origen y MITRE ATT&CK.
`AlertEvidence`	Entidades/evidencias asociadas a una alerta: usuario, dispositivo, archivo, IP, URL, etc.

Ambas se relacionan mediante:

`AlertId`

1.1.1.2 Defender for Endpoint

Estas son las tablas más importantes:

Tabla	Qué buscar
`DeviceProcessEvents`	Creación y ejecución de procesos, command line, parent process.
`DeviceNetworkEvents`	Conexiones de red, IP, dominios, puertos.
`DeviceFileEvents`	Creación, modificación, eliminación y otras operaciones sobre archivos.
`DeviceRegistryEvents`	Creación/modificación del registro de Windows.
`DeviceLogonEvents`	Logons y autenticaciones realizadas en endpoints.
`DeviceEvents`	Eventos diversos de seguridad, especialmente Defender AV, ASR, exploit protection y otros controles.
`DeviceInfo`	Información del dispositivo: nombre, SO, estado, usuarios, etc.

Regla rápida:

Process → `DeviceProcessEvents`

Network → `DeviceNetworkEvents`

File → `DeviceFileEvents`

Registry → `DeviceRegistryEvents`

Logon → `DeviceLogonEvents`

1.1.1.3 Defender for Office 365

Tabla	Qué buscar
`EmailEvents`	Entrega, bloqueo, remitente, destinatario y detección de emails.
`EmailAttachmentInfo`	Archivos adjuntos de emails.
`EmailUrlInfo`	URLs contenidas en emails.
`EmailPostDeliveryEvents`	Acciones de seguridad posteriores a la entrega, como ZAP.

Ejemplo:

¿Quién recibió el phishing? → `EmailEvents`

¿Qué URL contenía? → `EmailUrlInfo`

1.1.1.4 Identidades

Tabla	Qué buscar
`IdentityLogonEvents`	Autenticaciones observadas por Defender for Identity y servicios Microsoft.
`IdentityDirectoryEvents`	Cambios y eventos en Active Directory.
`IdentityQueryEvents`	Consultas realizadas sobre objetos de Active Directory.
`IdentityInfo`	Información/contexto de identidades.
`EntraIdSignInEvents`	Sign-ins interactivos y no interactivos de Microsoft Entra ID.
`EntraIdSpnSignInEvents`	Sign-ins de service principals y managed identities.

Para recordar:

AD query → IdentityQueryEvents

AD modification → IdentityDirectoryEvents

1.1.1.5 Defender for Cloud Apps

Para actividades realizadas en aplicaciones cloud:

`CloudAppEvents`

Incluye actividad sobre:

- Microsoft 365;
- SharePoint/OneDrive;
- Exchange;
- otras cloud apps conectadas.

1.1.1.6 Microsoft Graph

Para llamadas realizadas contra Microsoft Graph API:

`GraphAPIAuditEvents`

Es especialmente útil para investigar:

- OAuth apps;
- service principals;
- abuso de Microsoft Graph;
- operaciones API sospechosas.

1.1.1.7 Defender for Cloud

Las tablas más importantes son:

- `CloudAuditEvents`: operaciones de control plane cloud;
- `CloudProcessEvents`: procesos en workloads cloud/containers;
- `CloudDnsEvents`: actividad DNS cloud;
- `CloudStorageAggregatedEvents`: actividad de almacenamiento cloud.

1.1.1.8 Cómo elegir la tabla en el examen

Necesito investigar...	**Tabla**
PowerShell ejecutado en endpoint	`DeviceProcessEvents`
Conexión a IP maliciosa	`DeviceNetworkEvents`
Archivo creado en endpoint	`DeviceFileEvents`
Cambio de registry para persistencia	`DeviceRegistryEvents`
Usuario que inició sesión en dispositivo	`DeviceLogonEvents`
Alertas High severity	`AlertInfo`
Entidades asociadas a una alerta	`AlertEvidence`
Email phishing recibido	`EmailEvents`
URL contenida en phishing	`EmailUrlInfo`
Adjunto de email	`EmailAttachmentInfo`
ZAP u otra acción post-delivery	`EmailPostDeliveryEvents`
Consulta LDAP/AD sospechosa	`IdentityQueryEvents`
Cambio de cuenta/grupo en AD	`IdentityDirectoryEvents`
Actividad de cloud app	`CloudAppEvents`
Llamada sospechosa a Microsoft Graph	`GraphAPIAuditEvents`

1.1.1.9 Schema reference

Si no se recuerda una tabla o columna, Advanced Hunting incluye:

Schema tree / Schema reference

que muestra:

- descripción de cada tabla;
- columnas;
- tipos de datos;
- posibles valores de `ActionType`;
- sample queries.

La disponibilidad de una tabla depende de:

qué servicios Defender/Sentinel estén desplegados y accesibles

Por ejemplo, sin Defender for Office 365:

```
EmailEvents
```

no devolverá la telemetría esperada.

1.1.2 Identify threats by using Kusto Query Language (KQL)

Advanced Hunting utiliza:

Kusto Query Language (KQL)

para buscar comportamientos sospechosos en la telemetría de Microsoft Defender XDR y Microsoft Sentinel.

Una consulta sigue normalmente:

Table → Filter → Transform/Aggregate → Output

Ejemplo:

```
DeviceProcessEvents
| where Timestamp > ago(24h)
| where FileName =~ "powershell.exe"
| project Timestamp, DeviceName,
ProcessCommandLine
| order by Timestamp desc
```

El operador:

|

encadena cada operación con el resultado de la anterior.

1.1.2.1 where

`where` = *filtrar filas*

Ejemplo:

```
DeviceProcessEvents
| where Timestamp > ago(1d)
| where FileName =~ "powershell.exe"
```

Los operadores básicos son:

`==`, `!=`, `>`, `<`, `>=`, `<=`

Para múltiples condiciones:

`and`, `or`, `not`

Ejemplo:

```
| where FileName =~ "powershell.exe"
and ProcessCommandLine has "DownloadString"
```

1.1.2.2 Tiempo

En hunting es fundamental limitar el intervalo:

```
| where Timestamp > ago(24h)
```

Funciones frecuentes:

- `ago(1h)`: hace una hora;
- `ago(7d)`: hace siete días;
- `between`: intervalo de valores.

Ejemplo:

```
| where Timestamp between
(datetime(2026-09-20) .. datetime(2026-09-21))
```

Para SC-200:

Aplicar el filtro temporal lo antes posible

reduce los datos procesados y mejora el rendimiento.

1.1.2.3 Operadores de strings

Los más importantes son:

Operador	Busca	Ejemplo
`has`	Término completo	`CommandLine has "powershell"`
`contains`	Subcadena	`CommandLine contains "power"`
`startswith`	Comienzo del texto	`FileName startswith "power"`
`endswith`	Final del texto	`FileName endswith ".exe"`
`in`	Uno de varios valores	`FileName in ("a.exe","b.exe")`

Microsoft recomienda generalmente:

`has` *antes que* `contains`

cuando se busca un término completo, porque suele ser más eficiente.

1.1.2.4 Case sensitivity

Por ejemplo:

`==` = case-sensitive

mientras que:

`=` $= case - insensitive$

Igualmente:

`in` = case-sensitive

`in~` = case-insensitive

Ejemplo típico:

```
| where FileName in~ ("powershell.exe",
"powershell_ise.exe")
```

1.1.2.5 project

`project` = *seleccionar las columnas que se muestran*

Ejemplo:

```
| project Timestamp, DeviceName,
FileName, ProcessCommandLine
```

También permite renombrar:

```
| project Time=Timestamp,
Device=DeviceName
```

1.1.2.6 extend

`extend` = *crear una columna calculada sin eliminar las existentes*

Ejemplo:

```
| extend Account =
strcat(AccountDomain, "\", AccountName)
```

Diferencia importante:

`project` → selecciona columnas de salida

`extend` → añade columnas calculadas

1.1.2.7 sort/order y top

Para ordenar:

```
| order by Timestamp desc
```

o:

```
| sort by Timestamp desc
```

Para devolver únicamente los primeros resultados:

```
| top 10 by Timestamp desc
```

1.1.2.8 limit / take

`limit`/`take` = *limitar número de filas*

Ejemplo:

```
DeviceProcessEvents
| take 100
```

No garantizan un orden concreto.
En cambio:

```
top 10 by Timestamp desc
```

sí selecciona resultados según un criterio de ordenación.

1.1.2.9 summarize

`summarize` = *agregardatos*

Ejemplo: contar procesos por dispositivo.

```
DeviceProcessEvents
| where Timestamp > ago(24h)
| summarize ProcessCount=count()
by DeviceName
```

Funciones especialmente relevantes:

- `count()`: número de eventos;
- `dcount()`: número aproximado de valores distintos;
- `countif()`: contar cuando se cumple una condición;
- `min()` / `max()`;
- `make`$_s et()$: $conjuntodevaloresdistintos$.

Ejemplo para detectar posible scanning SMB:

```
DeviceNetworkEvents
| where Timestamp > ago(12h)
| where RemotePort == 445
| summarize RemoteIPs=dcount(RemoteIP)
by DeviceName
| where RemoteIPs > 20
```

1.1.2.10 bin

Para agrupar eventos en intervalos temporales:

`bin()`

Ejemplo:

```
DeviceLogonEvents
| where Timestamp > ago(24h)
| summarize Failed=count()
by bin(Timestamp, 1h), DeviceName
```

Esto agrupa los eventos:

por bloques de una hora

y es útil para detectar picos o patrones.

1.1.2.11 join

`join` = *relacionarregistrosdedostablas*

Ejemplo conceptual:

```
AlertInfo
| where Severity == "High"
| join kind=inner (
AlertEvidence
) on AlertId
```

Así se combinan:

información de la alerta + entidades relacionadas

mediante una columna común:

`AlertId`

Para hunting es frecuente usar:

`join kind=inner`

cuando solo interesan registros con coincidencia en ambas tablas.

1.1.2.12 let

let = *guardarunvalorounaconsultareutilizable*

Ejemplo:

```
let suspiciousIP = "203.0.113.10";

DeviceNetworkEvents
| where Timestamp > ago(24h)
| where RemoteIP == suspiciousIP
```

Resulta especialmente útil para:

- IOC;
- listas;
- fragmentos de consulta reutilizables.

1.1.2.13 Datos dynamic

Algunas columnas contienen estructuras JSON de tipo:

dynamic

Puede utilizarse:

$$\texttt{parse}_j son()\texttt{parse}_j son()_{\texttt{parse}_j son()_{\texttt{parse}_j son()}}$$

para acceder a sus propiedades.
Ejemplo conceptual:

```
| extend Data=parse_json(AdditionalFields)
| project Data.SomeProperty
```

Las conversiones más frecuentes son:

`tostring()`, `toint()`, `todatetime()`

1.1.2.14 Patrón de hunting recomendado

Una consulta eficiente suele seguir:

```
Table
| where Timestamp > ago(...)
| where <specific filters>
| summarize / join / extend ...
| project <needed columns>
| order by ...
```

Microsoft recomienda especialmente:

- limitar primero por tiempo;
- filtrar lo antes posible;
- buscar sobre columnas concretas;
- preferir `has` frente a `contains` cuando sea posible;
- proyectar únicamente las columnas necesarias;
- evitar búsquedas globales innecesarias.

1.1.2.15 Ejemplo completo de hunting

Buscar PowerShell que utilice comandos frecuentes de descarga:

```
DeviceProcessEvents
| where Timestamp > ago(24h)
| where FileName in~ (
"powershell.exe",
"powershell_ise.exe")
| where ProcessCommandLine
has_any ("DownloadString",
"DownloadFile",
"WebClient")
| project Timestamp,
DeviceName,
AccountName,
ProcessCommandLine
| order by Timestamp desc
```

El razonamiento sería:

Process telemetry → last 24h → PowerShell → suspicious command → relevant columns

1.1.3 Create Advanced Hunting queries

Las consultas se crean desde:

Microsoft Defender portal → Hunting → Advanced hunting

Existen dos formas principales:

Modo	Uso
Guided mode	Construir consultas visualmente mediante filtros y condiciones, sin escribir KQL.
Advanced mode	Escribir directamente consultas KQL.

Para SC-200:

Guided = query builder

Advanced = KQL editor

1.1.3.1 Crear una query

El flujo habitual es:

Seleccionar datos → construir query → Run query → revisar resultados

En Advanced mode puede empezarse:

- desde cero;
- desde una sample query;
- desde una query previamente guardada;
- mediante **Go hunt** desde una entidad o evento.

1.1.3.2 Schema y ayuda

El editor proporciona:

- **Schema tree**;
- autocompletado;
- descripción de tablas y columnas;
- sample queries.

Por tanto, si se desconoce dónde está un dato:

consultar Schema tree / Schema reference

1.1.3.3 Go hunt

Desde determinadas entidades o eventos puede seleccionarse:

Go hunt

Defender genera automáticamente una consulta Advanced Hunting relacionada con esa entidad.
Después:

la query puede modificarse y ampliarse

como cualquier otra consulta.
Esto es útil para pivotar rápidamente desde:

- usuarios;
- dispositivos;
- archivos;
- IP;
- URLs;
- eventos de un incidente.

1.1.3.4 Guided mode

Guided mode permite seleccionar:

- data domain;
- filtros;
- condiciones;
- operadores lógicos.

Una query creada visualmente puede posteriormente:

abrirse en Advanced mode

para revisar o modificar el KQL generado.

1.1.3.5 Guardar consultas

Después de validar una query puede utilizarse:

Save query → Save as

y elegir:

- **My queries**: solo accesible para el usuario;
- **Shared queries**: accesible para otros analistas de la organización.

También existen:

Community queries

proporcionadas por Microsoft y la comunidad.

1.1.3.6 Múltiples consultas

Advanced Hunting permite abrir:

varias query tabs

dentro del mismo editor.

Microsoft recomienda utilizar estas tabs en lugar de múltiples pestañas del navegador para evitar perder consultas no guardadas.

1.1.3.7 Validar resultados

Tras:

Run query

deben comprobarse los resultados antes de reutilizar la consulta:

- si devuelve las entidades esperadas;
- si genera demasiado ruido;
- si el intervalo temporal es adecuado;
- si los filtros son suficientemente específicos.

Desde los resultados también puede:

- filtrar;
- pivotar hacia entidades;
- modificar la consulta;
- exportar resultados.

1.1.3.8 Flujo recomendado para SC-200

Hypothesis→ Choose table → Build query → Run → Validate → Refine → Save

1.1.3.9 Escenarios típicos de examen

Escenario 1: un analista no conoce KQL pero necesita crear una búsqueda.

Guided mode

Escenario 2: se quiere crear una consulta KQL personalizada.

Advanced mode

Escenario 3: desde una IP sospechosa de un incidente se quiere buscar rápidamente actividad relacionada.

Go hunt

Escenario 4: se quiere que otros analistas reutilicen la consulta.

Save as → Shared queries

1.1.4 Interpret threat analytics in Microsoft Defender XDR

Microsoft Defender XDR:

Threat analytics

proporciona informes de inteligencia elaborados por Microsoft sobre:

- threat actors y campaigns;
- vulnerabilidades;
- malware;
- técnicas de ataque;
- amenazas emergentes.

Lo importante es que combina:

Microsoft threat intelligence + datos del propio entorno

para determinar si una amenaza es relevante para la organización.

1.1.4.1 Threat analytics dashboard

El dashboard prioriza principalmente:

- **Latest threats**: informes más recientes.
- **High-impact threats**: amenazas con mayor impacto actual en la organización.
- **Highest exposure threats**: amenazas frente a las que la organización presenta mayor exposición.

Para SC-200:

Impact $\neq$ Exposure

Impact $\rightarrow$ alertas/incidentes que ya afectan al entorno

Exposure $\rightarrow$ vulnerabilidades o configuraciones que podrían permitir el ataque

1.1.4.2 Threat analytics report

Un informe contiene principalmente:

- **Overview**;
- **Analyst report**;
- **Related incidents**;
- **Impacted assets**;
- **Endpoints exposure**;
- **Recommended actions**;
- **Indicators**.

1.1.4.3 Overview

Permite responder rápidamente:

¿Esta amenaza afecta o podría afectar a mi organización?

Puede mostrar:

- active/resolved alerts;
- incident severity;
- impacted assets;
- exposición de endpoints.

1.1.4.4 Related incidents

Related incidents

muestra incidentes del entorno asociados a la amenaza.

1.1.4.5 Impacted assets

Muestra activos relacionados con alertas de la amenaza:

- devices;
- users;
- mailboxes;
- apps;
- cloud resources.

Sirve para determinar:

scope de la amenaza

1.1.4.6 Endpoints exposure

La pestaña:

Endpoints exposure

indica qué dispositivos son vulnerables a las debilidades aprovechadas por la amenaza.

El nivel de exposición depende principalmente de:

- severidad de las vulnerabilidades/misconfigurations;
- número de dispositivos afectados.

Por tanto:

Exposure no significa necesariamente compromise

Un dispositivo puede estar expuesto aunque todavía no exista ninguna alerta relacionada.

1.1.4.7 Recommended actions

Incluye medidas para reducir la exposición, por ejemplo:

- instalar security updates;
- corregir configuraciones;
- habilitar protecciones;
- aplicar mitigaciones.

Las recomendaciones monitorizadas aparecen con su:

status

para comprobar qué mitigaciones ya se han aplicado.

Importante: **Analyst report puede incluir mitigaciones adicionales no monitorizadas automáticamente**

1.1.4.8 Analyst report

Es el análisis detallado realizado por investigadores de Microsoft.

Para threat hunting es especialmente importante:

Detections / Hunting queries

porque Microsoft proporciona consultas listas para buscar posibles rastros de esa amenaza.

Estas queries pueden ejecutarse en:

Advanced Hunting

y, cuando corresponda, en Microsoft Sentinel.

1.1.4.9 Indicators

La pestaña:

Indicators

puede proporcionar IOC asociados a la amenaza, como:

- IP addresses;
- domains;
- URLs;
- hashes.

Pueden utilizarse para:

investigation + hunting + remediation

1.1.4.10 Escenarios típicos de examen

Escenario 1: se quiere saber qué amenazas ya están generando alertas importantes en la organización.

High-impact threats

Escenario 2: se quiere priorizar una amenaza porque muchos dispositivos son vulnerables aunque todavía no haya compromiso.

Highest exposure threats / Endpoints exposure

Escenario 3: se necesita saber qué usuarios o dispositivos están afectados.

Impacted assets

Escenario 4: se quieren buscar proactivamente rastros de una nueva campaña.

Analyst report → Hunting queries → Advanced Hunting

Escenario 5: se quieren conocer las medidas concretas para reducir el riesgo.

Recommended actions + Analyst report

1.1.5 Create hunting graphs, including blast radius

Microsoft Defender permite representar visualmente relaciones entre entidades mediante:

Hunting graphs

desde:

Hunting → Advanced hunting → Hunting graph

El grafo utiliza:

- **Nodes**: entidades como usuarios, dispositivos, IPs o recursos cloud.
- **Edges**: relaciones entre esas entidades.

Su objetivo es descubrir relaciones y rutas de ataque que pueden ser difíciles de identificar únicamente con resultados tabulares KQL.

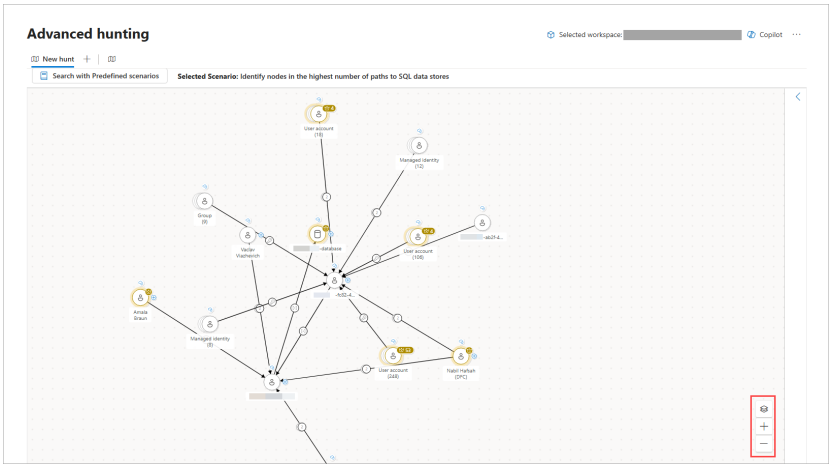


Figure 1: Caption

1.1.5.1 Crear un hunting graph

Puede crearse mediante:

Advanced hunting → Hunting graph icon

o:

Create new → Hunting graph

El grafo puede construirse utilizando:

Predefined scenarios

que ejecutan consultas preparadas para casos comunes.
Por ejemplo:

Attack paths to critical asset

permite identificar posibles rutas hacia un activo crítico.
Los resultados pueden filtrarse por características como:

- critical/vulnerable assets;
- internet exposure;
- sensitive data;
- tipo y dirección de las relaciones.

1.1.5.2 Explorar el grafo

Al seleccionar un nodo o relación puede:

- consultar información de la entidad;
- expandir entidades relacionadas;
- identificar rutas hacia otros activos;
- localizar puntos críticos o *choke points*.

Por tanto:

Hunting graph = explorar relaciones y posibles rutas de ataque

1.1.5.3 Blast radius

Durante la investigación de un incidente puede utilizarse:

View blast radius

sobre una entidad del attack graph.
Blast radius determina:

hasta dónde podría propagarse el compromiso desde una entidad afectada

y muestra posibles rutas hacia activos críticos.
Ejemplo:

Compromised user → Device → Privileged identity → Critical server

Combina:

- **post-breach information**: lo que ya ha ocurrido;
- **pre-breach exposure**: rutas que todavía podría explotar el atacante.

1.1.5.4 Interpretar Blast radius

Al seleccionar:

View blast radius

Defender muestra las rutas de mayor riesgo desde la entidad seleccionada hacia posibles objetivos.
Inicialmente muestra:

las 8 attack paths mejor clasificadas

y:

View full blast radius list

permite consultar el resto de objetivos alcanzables.
Blast radius:

extiende y sustituye a Attack path analysis

en la experiencia actual.

1.1.5.5 Uso operativo

Blast radius ayuda a decidir:

- qué entidades investigar primero;
- qué cuentas o dispositivos contener;
- qué activos críticos están potencialmente en riesgo;
- qué ruta debe romperse para impedir la propagación.

Por tanto:

Blast radius → priorizar investigación y contención

1.1.5.6 Escenarios típicos de examen

Escenario 1: se quieren visualizar relaciones entre usuarios, dispositivos y recursos durante un hunt.

Hunting graph

Escenario 2: se quiere conocer por qué rutas podría alcanzarse un activo crítico.

Predefined scenario → Attack paths to critical asset

Escenario 3: un usuario está comprometido y se quiere conocer qué otros activos podrían verse afectados.

View blast radius

1.1.6 Analyze relationships between entities by using Sentinel Graph

Microsoft Sentinel Graph es la capacidad de análisis de grafos de Sentinel para modelar y consultar relaciones entre:

- identidades;
- dispositivos;
- recursos cloud;
- actividades;
- datos;
- threat intelligence.

Se accede desde:

Microsoft Sentinel → Graphs

Su objetivo es responder preguntas difíciles de resolver únicamente mediante datos tabulares, por ejemplo:

¿Qué recursos puede alcanzar un atacante si compromete esta identidad?

1.1.6.1 Graph Query Language

Sentinel Graph se consulta mediante:

Graph Query Language (GQL)

y no mediante KQL.
Antes de consultar puede revisarse:

Schema

para conocer:

- tipos de entidades;
- relaciones disponibles;
- propiedades de entidades y relaciones.

Para SC-200:

KQL = eventos/tablas

GQL = entidades/relaciones

1.1.6.2 Identity attack graph

Uno de los grafos predefinidos más importantes es:

Identity attack graph

que representa cómo las identidades se relacionan con recursos Azure mediante:

- permissions;
- group memberships;
- trust relationships.

Permite identificar:

lateral movement y privilege escalation paths

Por ejemplo:

User → Group → Privileged role → Critical Azure resource

Esto permite priorizar la corrección de las rutas más cortas o peligrosas hacia activos críticos.

1.1.6.3 Explorar relaciones

Desde un resultado del grafo puede:

- seleccionar una entidad y revisar sus propiedades;
- expandir entidades relacionadas;
- seguir relaciones en distintas direcciones;
- identificar rutas hacia activos críticos;
- cambiar a **Table view** para validar los resultados.

También existen:

predefined GQL queries

para escenarios frecuentes, por lo que no siempre es necesario escribir GQL desde cero.

1.1.6.4 Custom graphs

Sentinel permite crear:

Custom graphs

actualmente en:

Preview

utilizando datos del:

Microsoft Sentinel Data Lake

y también fuentes externas.
El flujo es:

Data Lake / external data

⇓

Jupyter Notebook

⇓

Publish / materialize graph

⇓

Query with GQL

Los grafos se materializan mediante:

Graph jobs

que pueden ejecutarse:

- on-demand;
- scheduled.

Un graph job on-demand conserva el grafo durante:

30 días

mientras que uno scheduled reconstruye el grafo según la programación configurada.

1.1.6.5 Escenarios típicos de examen

Escenario 1: se quiere conocer qué recursos Azure puede alcanzar una identidad comprometida mediante permisos y grupos.

Identity attack graph

Escenario 2: se quiere consultar relaciones entre entidades dentro de Sentinel Graph.

GQL

Escenario 3: se necesita combinar información del Sentinel Data Lake y fuentes externas para modelar relaciones propias.

Custom graph

1.2 Detect threats by using the Microsoft Sentinel platform

1.2.1 Create and monitor hunting queries

Microsoft Sentinel utiliza:

Hunting queries

para buscar proactivamente comportamientos sospechosos que todavía no han generado alertas.

Se administran desde:

Microsoft Sentinel → Threat management → Hunting → Queries

1.2.1.1 Tipos de hunting queries

Existen principalmente:

- **Built-in queries**: instaladas mediante soluciones del Content Hub.
- **Custom queries**: creadas por el analista.

Las built-in pueden:

Clone

para crear una versión personalizada.

1.2.1.2 Crear una hunting query

Desde:

Hunting → Queries → New query

se configuran principalmente:

- Name;
- Description;
- **KQL query**;
- **Entity mappings**;

Los:

Entity mappings

permiten convertir columnas de la query en entidades como:

- Account;
- Host;
- IP;
- URL;
- File.

Esto facilita continuar posteriormente la investigación.

1.2.1.3 Ejecutar y monitorizar queries

Desde la pestaña Queries pueden seleccionarse una o varias consultas y ejecutar:

Run selected queries

Sentinel permite monitorizar especialmente:

- **Results**: número de resultados;
- **Results delta**: cambio respecto a la ejecución anterior;
- **Results delta percentage**: variación porcentual.

Por tanto, un:

spike en Results / Results delta

puede indicar nueva actividad sospechosa que merece investigación.
Las queries pueden filtrarse por:

- data source;
- technique;
- resultados;
- favorites.

1.2.1.4 View query results

Cuando una query devuelve actividad interesante:

View query results

abre los resultados para investigar los eventos concretos mediante KQL.
Desde ahí pueden analizarse:

- entidades;
- timestamps;
- IOC;
- actividad relacionada.

1.2.1.5 Bookmarks

Los resultados especialmente relevantes pueden conservarse como:

Bookmarks

Un bookmark conserva:

- resultado seleccionado;
- query que lo generó;
- intervalo temporal;
- notes;
- tags;
- entity mappings;
- MITRE ATTCK mappings.

Actualmente existe una diferencia importante:

Defender portal: permite visualizar bookmarks existentes

Azure portal: permite crear nuevos bookmarks

Los bookmarks tampoco están disponibles en la experiencia unificada de:

Advanced Hunting

1.2.1.6 Hunts

Para una investigación estructurada puede crearse un:

Hunt

actualmente en Preview.
Un Hunt permite organizar:

Hypothesis → Queries → Findings → Conclusion

y registrar:

- queries;
- bookmarks;
- entities;
- comments;
- estado de la hipótesis.

Es útil para campañas de hunting que requieren seguimiento durante varias sesiones.

1.2.1.7 Convertir hallazgo en detección

Una Hunting Query es:

proactiva y ejecutada durante hunting

No constituye por sí sola una detección automática continua.
Si una query descubre un comportamiento que debe monitorizarse continuamente:

New alert rule → Create Microsoft Sentinel alert

para convertir la lógica en una:

Analytics Rule

Conceptualmente:

Hunting query → hallazgo repetible → Analytics Rule → alerta automática

1.2.1.8 Escenarios típicos de examen

Escenario 1: se quiere buscar proactivamente actividad que todavía no genera alertas.

Hunting query

Escenario 2: una query muestra un aumento repentino de coincidencias.

Revisar Results delta / Results delta percentage

Escenario 3: se encuentra un evento sospechoso que debe conservarse para continuar la investigación.

Bookmark

Escenario 4: una query identifica una técnica que debe detectarse automáticamente en el futuro.

Create Microsoft Sentinel alert / Analytics Rule

Escenario 5: se quiere organizar una investigación basada en una hipótesis durante varias sesiones.

Hunt

1.2.2 Create and manage KQL jobs in Data lake

Los:

KQL jobs

ejecutan consultas KQL de forma:

- **one-time**;
- **scheduled**;

sobre datos almacenados en el:

Microsoft Sentinel Data Lake

y almacenan los resultados en una tabla para poder reutilizarlos posteriormente. Son especialmente útiles para:

- investigaciones históricas;
- IOC matching retrospectivo;
- baselines y anomaly detection;
- agregaciones sobre grandes volúmenes;
- promover datos históricos al Analytics tier.

1.2.2.1 Crear un KQL job

Puede crearse desde:

Microsoft Sentinel → Data lake exploration → KQL queries

mediante:

Create job

o desde:

Data lake exploration → Jobs → Create new KQL job

El flujo básico es:

$KQLquery\rightarrow$ Destination workspace/table → One-time o Schedule → Run

1.2.2.2 Destination table

Los resultados pueden escribirse en:

- una **nueva tabla**;
- una **tabla existente**.

Y la tabla destino puede pertenecer al:

Analytics tier o Data Lake tier

Si se promueven resultados a una nueva tabla del Analytics tier, la tabla creada utiliza el sufijo:

`_KQL_CL`

Si se crea directamente una tabla en el Data Lake mediante System tables:

`_KQL`

Esta última opción está actualmente en:

Preview

1.2.2.3 Promote to Analytics tier

Uno de los usos más importantes es:

Data Lake → KQL job → Analytics tier

Esto permite tomar un subconjunto de datos históricos y llevarlo al Analytics tier para utilizarlo posteriormente en capacidades de Sentinel que necesitan datos Analytics.

Por ejemplo:

12 meses de logs históricos → buscar actividad de una IP maliciosa
→ promover matches al Analytics tier

1.2.2.4 One-time vs. Scheduled

Tipo	Uso
One-time	Ejecutar una investigación histórica puntual.
Scheduled	Ejecutar periódicamente la consulta para crear baselines, agregaciones o hunting continuo.

Un:

One-time job

se ejecuta inmediatamente después de crearlo.

Un Scheduled job permite definir:

- fecha/hora de inicio;
- recurrencia;
- fecha de finalización o ejecución indefinida.

1.2.2.5 Schema

Si el resultado se añade a una tabla existente:

el schema de la query debe coincidir con el schema de la destination table

Si las columnas o tipos no coinciden:

el job falla.

1.2.2.6 Casos típicos de examen

Escenario 1: se necesita investigar una IP descubierta hoy en logs conservados durante varios meses en Data Lake.

One-time KQL job

Escenario 2: se quiere crear diariamente un baseline de actividad de inicio de sesión.

Scheduled KQL job

Escenario 3: se encuentra información histórica relevante y se quiere utilizar en capacidades del Analytics tier.

KQL job → destination table en Analytics tier

Escenario 4: un scheduled job deja de producir resultados.

Jobs → View history / Recent health issues

1.2.3 Create and manage Summary rule tables for querying

Las:

Summary Rules

ejecutan periódicamente una consulta KQL sobre grandes volúmenes de logs y almacenan el resultado agregado en una:

custom table del Analytics tier

El objetivo es consultar después un conjunto mucho más pequeño y rápido que los logs originales.

Conceptualmente:

Raw high-volume logs → Summary Rule → Aggregated custom table
→ Fast queries / Analytics Rules

1.2.3.1 Casos de uso

Son especialmente útiles para:

- logs de firewall o red de gran volumen;
- estadísticas y tendencias;
- threat hunting sobre largos periodos;
- reducir el volumen que debe mantenerse en Analytics;
- generar datos agregados que posteriormente utilizarán Analytics Rules.

Por ejemplo:

> 1 TB/día de firewall logs → resumir IP + conexiones + FirstSeen/LastSeen → tabla pequeña en Analytics

1.2.3.2 Crear una Summary Rule

Ruta:

> **Microsoft Sentinel → Configuration → Summary rules → Create**

Debe configurarse principalmente:

- Name;
- Description;
- **Destination table**;
- **KQL summary query**;
- **Query frequency**;
- optional execution delay;
- start time.

La tabla destino puede ser:

- una custom table existente;
- una nueva custom table.

Una nueva tabla utiliza:

`<TableName>_CL`

1.2.3.3 Ejemplo

```
CommonSecurityLog
| summarize Connections=count(),
FirstSeen=min(TimeGenerated),
LastSeen=max(TimeGenerated)
by SourceIP, DestinationIP
```

En lugar de consultar millones de eventos originales, posteriormente se consulta directamente:

la tabla resumen

con una cantidad mucho menor de registros.

1.2.3.4 Summary Rule vs. KQL Job

Esta diferencia es muy importante:

Función	Uso principal
Summary Rule	Agregación frecuente de grandes volúmenes para crear una tabla Analytics optimizada.
KQL Job	Investigación histórica/ad-hoc o scheduled sobre Sentinel Data Lake, con consultas más complejas.

Por tanto:

Frequent aggregation → Summary Rule

Historical investigation → KQL Job

1.2.3.5 Escenarios típicos de examen

Escenario 1: se ingieren enormes cantidades de firewall logs en Auxiliary y se necesita analizarlos rápidamente.

Summary Rule → Analytics summary table

Escenario 2: se quiere ejecutar cada hora una agregación de IPs y conservar solo estadísticas relevantes.

Summary Rule

Escenario 3: se necesita generar alertas utilizando datos derivados de logs muy voluminosos.

Summary Rule → Summary table → Analytics Rule

Escenario 4: se quiere comprobar por qué una Summary Rule ha fallado.

View run history / SummaryLogs

1.2.4 Hunt for threats by using Notebooks, including connection to the Sentinel MCP Server

Microsoft Sentinel permite utilizar:

Jupyter Notebooks

cuando una investigación requiere más capacidades que una consulta KQL convencional.

Son especialmente útiles para:

- Machine Learning;
- detección de anomalías;
- análisis estadístico;
- correlación de grandes datasets;
- visualizaciones avanzadas;
- integración con fuentes externas;
- detección de ataques *low-and-slow*.

1.2.4.1 Notebooks

Los notebooks combinan:

Python + KQL + librerías de análisis

permitiendo mantener en un único documento:

- código;
- consultas;
- resultados;
- visualizaciones;
- documentación de la investigación.

Esto los hace especialmente apropiados para investigaciones complejas y reutilizables.

1.2.4.2 MSTICPy

Una librería especialmente importante es:

MSTICPy

desarrollada para threat hunting e investigación.
Proporciona funciones para:

- consultar datos de seguridad;
- enrichment con Threat Intelligence;
- geolocalización de IP;
- extracción de IOC;
- análisis de anomalías;
- timelines y visualizaciones.

También puede utilizarse:

Kqlmagic

para ejecutar consultas KQL directamente desde celdas del notebook.

1.2.4.3 Sentinel Data Lake notebooks

Los notebooks actuales del Sentinel Data Lake pueden utilizar:

Python + Apache Spark

para analizar grandes volúmenes de datos históricos.
Son especialmente útiles cuando:

KQL tradicional no es suficiente para el análisis requerido.

Los notebooks pueden además ejecutarse de forma programada como:

Notebook jobs

y escribir sus resultados en custom tables del Data Lake o Analytics tier.

1.2.4.4 Microsoft Sentinel MCP Server

Microsoft Sentinel también proporciona soporte para:

Model Context Protocol (MCP)

que permite a clientes y agentes de IA acceder a herramientas de Sentinel mediante una interfaz estándar.

El servidor MCP permite utilizar lenguaje natural para:

- localizar tablas relevantes;
- consultar el Sentinel Data Lake;
- ejecutar KQL;
- analizar entidades;
- explorar relaciones mediante Sentinel Graph;
- apoyar threat hunting e incident triage.

1.2.4.5 Data Exploration collection

La colección principal para hunting es:

Microsoft Sentinel Data Exploration MCP

que incluye herramientas para:

- Semantic search sobre el catálogo de tablas;
- ejecutar consultas KQL;
- listar workspaces;
- **Entity Analyzer**;
- análisis mediante Sentinel Graph.

El endpoint actual es:

`https://sentinel.microsoft.com/mcp/data-exploration`

1.2.4.6 Conexión al MCP Server

El MCP Server puede conectarse a clientes compatibles como:

- Microsoft Security Copilot;
- Microsoft Copilot Studio;
- Microsoft Foundry;
- Visual Studio Code.

El flujo conceptual es:

> AI/MCP client → Microsoft Sentinel MCP Server → Data Lake / Sentinel Graph

La autenticación se realiza mediante:

> **Microsoft Entra ID**

y el usuario, managed identity o service principal debe disponer como mínimo de un rol compatible, como:

- Security Reader;
- Security Operator;
- Security Administrator.

1.2.4.7 Notebooks + MCP

Son capacidades complementarias:

> **Notebook = análisis programático**

> **MCP = acceso mediante herramientas/IA y lenguaje natural**

Por ejemplo:

> Notebook → analiza grandes volúmenes → escribe resultados en custom table
>
> ⇓
>
> Sentinel MCP → consulta esos resultados mediante lenguaje natural

Por tanto, el MCP Server:

> **no sustituye al notebook**

sino que proporciona otra forma de acceder y razonar sobre los datos de Sentinel.

1.2.4.8 Escenarios típicos de examen

Escenario 1: se necesita aplicar Machine Learning y visualizaciones avanzadas durante una investigación.

Jupyter Notebook

Escenario 2: se necesita enrichment, IOC extraction y análisis de anomalías desde Python.

MSTICPy

Escenario 3: se quieren ejecutar consultas KQL dentro de un notebook.

Kqlmagic

Escenario 4: un analista quiere consultar los datos del Sentinel Data Lake mediante lenguaje natural.

Sentinel MCP Server – Data Exploration

Escenario 5: se necesita analizar relaciones entre entidades mediante IA.

Sentinel MCP → Graph tools